POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Digital Twin of a Triplex Reciprocating Pump: anomaly detection, fault classification and model updating by MCMC-MH

Digital Twin for Health and Usage Monitoring, 2024/25

Pietro Saggionetto 11003274

Introduction

The target of the activity is to perform diagnosis, so anomaly detection, fault classification and model updating by MCMC-MH on a triplex reciprocating pump. The physical model of the pump isn't available but a very detailed Simulink model (developed by Mathworks) is given in order to obtain the needed signals from the sensors acquisition. Different sensors are available in the model but the focus will be mostly on the use of the output flow rate sensor (but not only). Three different faults has been implemented in the model which are the ones the activity refers to, in particular are: bearing fault/wear, sealing leakage at the plunger head and blocked inlet. In order to perform anomaly detection Mahalanobis distance will be exploited; with regard to the fault classification ANN algorithms will be used and finally, for model updating the MCMC-MH method will be applied by the use of surrogate modeling also. Here following is shown a simple pump like the one which will be analyzed in the activity of this report.

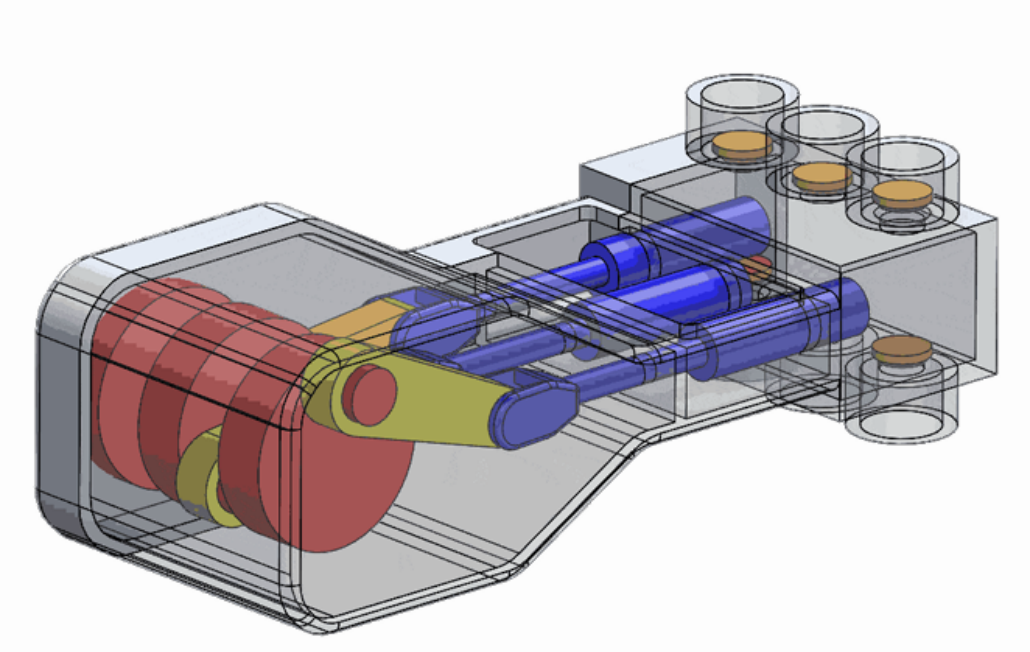

Figure (1): Structure of a triplex reciprocating pump

1. Model, Analysis of the sensor signals and Feature Extraction

In this paragraph will be analyzed the Simulink model of the pump, showing the main features, the different component that compose it and how the faults has been modeled. Furthermore will be shown the outputs which will be exploited for the following analysis and how to treat them.

1.1. Model

Actually, in order to model all the components the Simscape library, and so implicit modeling, has been exploited. The entire Simulink model can be seen in Figure (2). Three different domains can be distinguished:

- The **electrical domain** which includes the DC voltage source, the current sensor and the electric motor.
- The **mechanical domain** which includes the plungers and all the kinematics involved in modeling the pump.
- The **hydraulic domain** including the working fluids and the load.

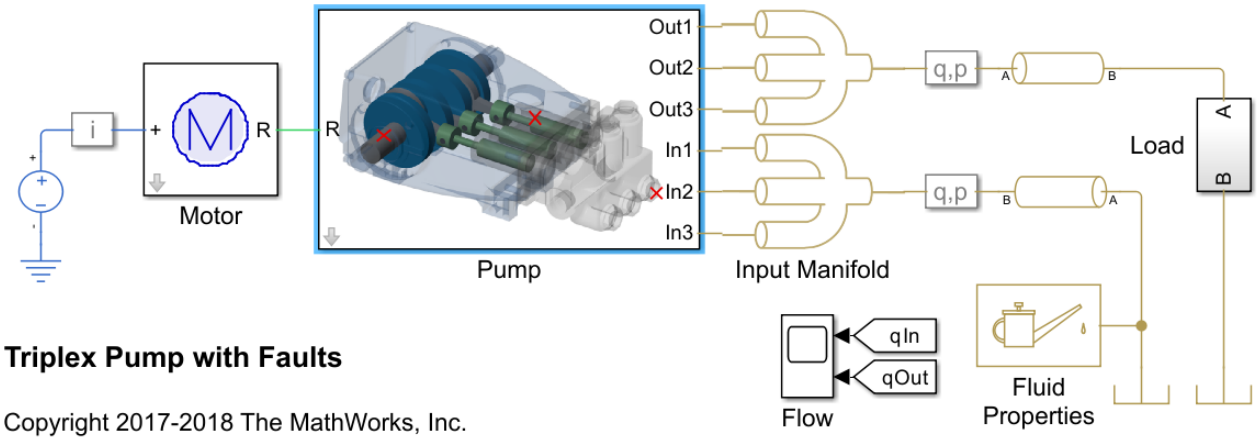

Figure (2): Simulink model of the pump

Going more into the details, lets start analyzing the different domains separately and all the components which compose it.
Starting from the electrical domain, shown in Figure (3), it already has been said that it is composed by three different component. The DC voltage source is a component coming from the Simscape library which provides a current.

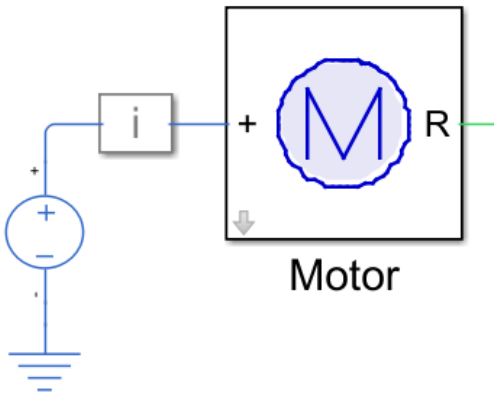

Figure (3): Electrical domain of the model

The current sensor model is shown in the next figure. Actually uses a simple current sensor coming from the Simscape library in which, by the way, the measured signal of the current is disturbed by adding some noise to it, in specific white noise. Anyway, since it might be required to perform some oversampling on the dataset which will be built it is convenient to log the signals (in general) without noise as weel.

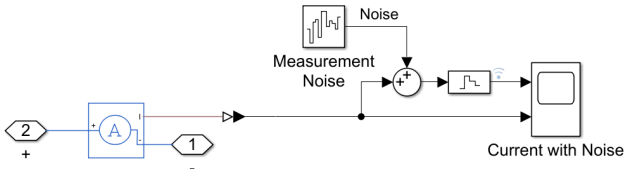

Figure (4): Current sensor

The motor subsystem, shown in the next figure. features an electrical connection as input and a mechanical connection as output. The Simscape motor component convert the electrical power into mechanical one. The provided torque is then connected to a gearbox which reduced the *rpm* with a gear ratio of 2. The actual speed of the motor needs to be controlled since it is attached to a non constant load as can be a pump. To do such a PI speed controller is present which acts on the positive pin of the electric motor.

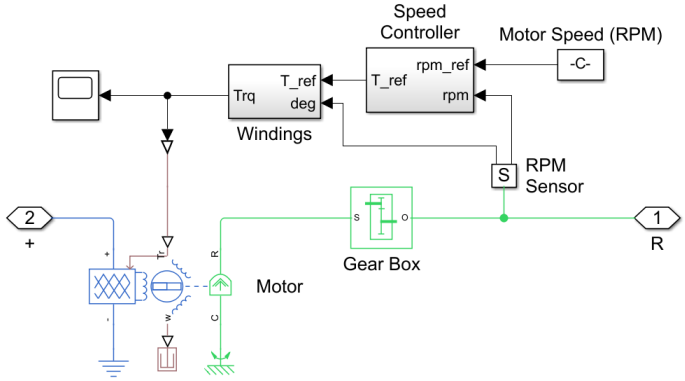

Figure (5): Electric motor model

The Pump subsystem is the main component of the model, and the most complex one. It includes both mechanical and hydraulic domain since it is connected to the motor on one side, and to the inlet and outlet manifolds on the other side. It includes the modeling of all the three faults which will be considered in this activity.
Below can be seen the image representing the Pump system. It is made up of many components. In particular those are a housing, a crankshaft and three plungers. The housing and the crankshaft models the kinematics and the dynamics of the mechanical domain, the plungers instead connects the mechanical domain to the hydraulic domain. Here can be also shown how the bearing wear is modeled. Actually to do such it is needed to act on the revolute crank component. This represents the connection between the housing and the crankshaft, and though the bearing. To actually model the bearing wear what is done is modifying the dumping coefficient of such component.

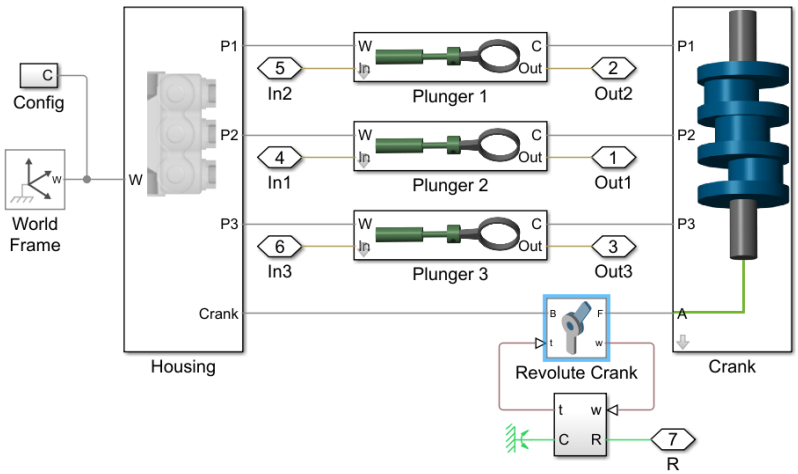

Figure (6): Pump model

A detailed view on how the plunger is modeled is given in Figure (29). Here are modeled both the kinematics and the hydraulics of the system. The mechanical and hydraulic domains are connected through the hydraulic actuator component which actually drive the hydraulic volume through the kinematics of the mechanical part, which is in turn driven by the motor. The hydraulic domain features two check valves to block the fluid in the discharge and suction phase respectively. Here actually are modeled two of the three possible faults that has been implemented.
For the seal leak a variable area orifice is present, which involves the flow for both discharge and suction phase of the pump. To model the blocked inlet instead, the maximum passage area value of the check valve is changed.

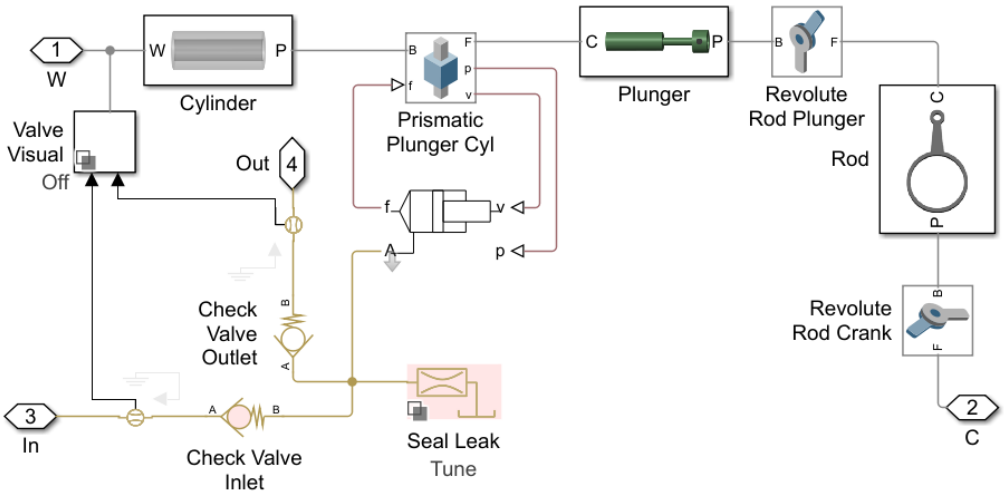

Figure (7): Plunger model

Concluding the model explanation lets focus on the external hydraulic domain which is shown in the following image. It is composed by two manifolds, one for the inlet and one for the outlet respectively which split and merges the flow for the three cylinders. There are then two pressure and flow sensors, one for the inlet and one for the outlet, two pipeline blocks and a load block.

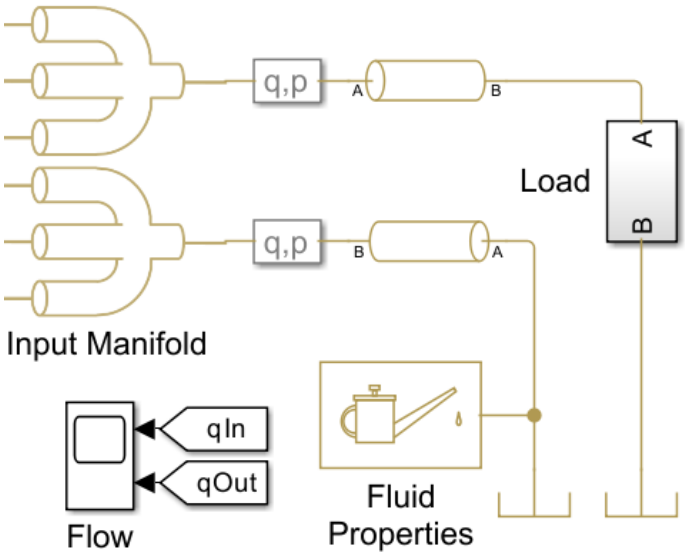

Figure (8): Hydraulic domain in the model

Following are shown more into detail both the flow/pressure sensor for the output flow (the one for the inlet is the same) and the load block.
In the sensor it can be seen that are used simple Simscape flow rate sensor and Pressure sensor. As we already saw with the current sensor white noise is added for both the measurements. The load block instead is simply modeled as an orifice and a pressure source.

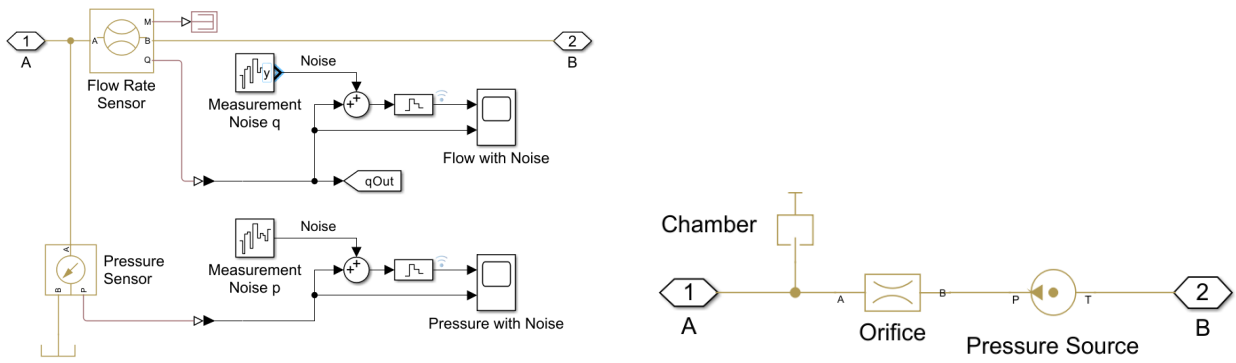

Figure (9): Flow rate and pressure sensor model n the left and load block model on the right

1.2. Sensor signal analysis

The sensors which are installed in the the system are six in total and they measure the following features:

- The pump output pressure.
- The pump output flow rate.
- The pump input pressure.
- The pump input flow rate.
- The motor speed.
- The motor current.

The sensor will be used to perform the activity described in this report is the output flow rate one since the given codes already implements it (in a following section also another sensor will be used). Actually what one should do to select the best sensor, is a sensitivity analysis for each of them with each fault and then select the more affected one.
To have an idea of the signals with which one have to deal four simulation have been performed: one with no damage, one with a medium-sized leak, one with a medium sized blockage and one with a medium-sized bearing wear. The resulting output flow rate signal (without noise) comparing the undamaged situation with each of the faults are shown in the following pictures.

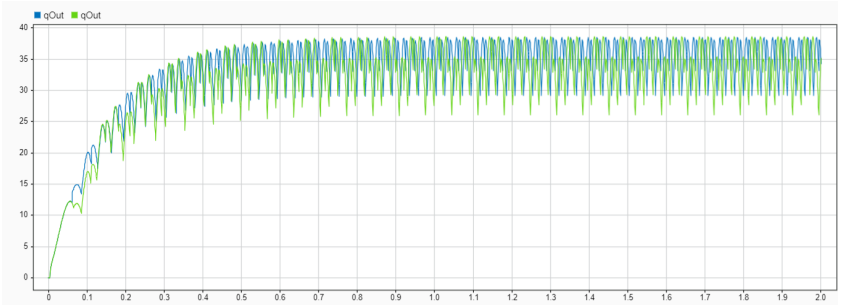

Figure (10): Comparison between undamaged and leak damage case signals

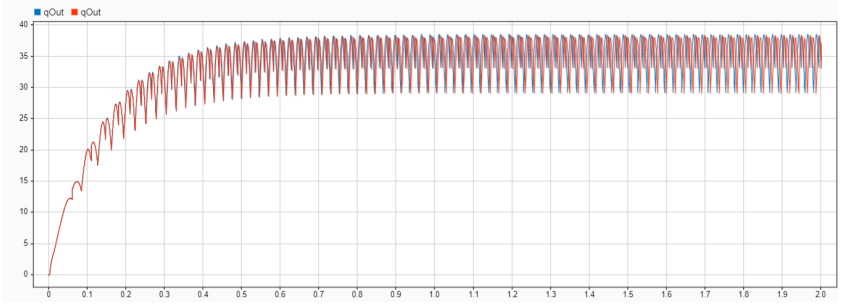

Figure (11): Comparison between undamaged and blocked damage case signals

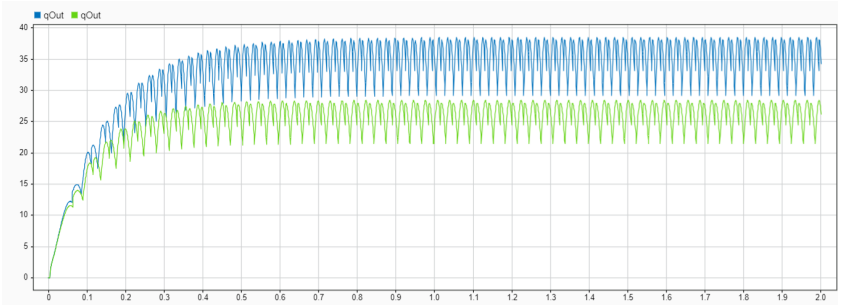

Figure (12): Comparison between undamaged and bearing damage case signals

By looking to the images it is possible to notice some differences between the signals for each different case, so the signal contains information of the damage which is present (or not). Anyway the signal is very complex and difficult to be approximated through surrogate models. Since those will be needed later a solution to characterize the signals in an accurate way is needed. What can be done in this regards is extracting $features$, or better statistical indicators, from the signals in the time domain or a transformed version of those signals. Those signal features should be representative of the system condition which is related to them. Usually a single feature is not enough to classify a system condition, but a combination of features may be.
Since selecting features from the signal is not a simple practice there gonna will be used the one already selected by Mathworks (which is the creator of the model). In the end the features that will be extracted are the following 14:

- The power in the $10 - 20Hz$ range.
- The power in the $40 - 60Hz$ range.
- The power in the $100 - 250Hz$ range.
- The frequency of the spectral kurtosis peak.
- The signal cumulative sum range.
- The signal mean, variance, kurtosis, peak2peak, peak magnitude to RMS, RMS, MAD.

2. Anomaly Detection and Fault Classification

In this paragraph what will be done is to perform anomaly detection first and subsequently fault classification on the pump. This will be done exploiting the Mahalanobis distance concept for the first task and ANN for the second task.

2.1. Anomaly detection

The target of this part of the activity is to perform anomaly detection. Even though it has been used only one sensor signal in order to perform this, which is the output flow rate, the case falls into the multivariate case. This because to perform the anomaly identification it is needed to rely on the features of the used signal, which have been shown in the previous paragraph they are 14 in total. What actually changes compared to the single variable case is the used formula which, in this case, is in matrix form. Lets though explain the concept of Mahalanobis distance.
In the context of Gaussian distribution an outlier is a data point that differs significantly from other observations. That "significantly" needs to be somehow quantified. What are needed though are a baseline condition which describes the healthy state, and a distance parameter required to evaluate the distance of a new observation from the baseline condition. This "distance parameter" in the end is the Mahalanobis distance which, specifically, allows to calculate the statistical distance of a vector of n components x (in this case the number of features) from a multivariate Gaussian distribution with a mean vector $\overline{\mu_\epsilon}$ and a covariance matrix S. In the end it reads as:

$$M_D(t) = \sqrt{(x - \overline{\mu_\epsilon})^T S^{-1} (x - \overline{\mu_\epsilon})}$$

where $\overline{\mu_\epsilon}$ and S represents the baseline, so the mean and covariance of the undamaged samples.
Furthermore a threshold of Mahalanobis distance value above which the data is considered as "anomaly" must be defined. Such threshold will be based on the baseline statistics. It is also important to consider that an assumption to apply this is the parameters considered must be normally distributed, so this needs to be verified on the baseline datas.
Starting now to talk about what has been practically done, the first thing was to asses the Mahalanobis distance for the undamaged case, in order to choose a threshold value. For the baseline has been considered 1000 as well for the test samples. Those samples has been derived by running the model with the undamaged condition parameters, and then processing the signal related to the output flow rate in order to create 1000 different samples adding Gaussian noise starting from the same original signal coming from the simulation. This process has been done two times in order to derive 1000 samples for the baseline and 1000 sample for the test. For all of those samples has been extracted the features and then those have been submitted to the Jarque-Bera test to verify whether or not they were normally distributed. From the test it actually came out that two of the 14 features are not normally distributed and though needs to be discarded, which are fPeak and pKurtosis. After having done this the Mahalanobis distance can be applied to the test set by using the mean and the covariance of the baseline set. The result coming from this is shown in the following image.

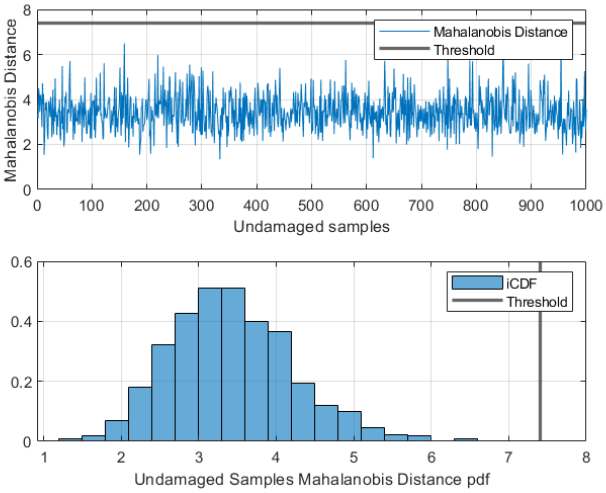

Figure (13): Mahalanobis distance of the undamaged samples

Based on this the chosen threshold is 7.4 as can be also noticeable in the image above. To test the performance of the anomaly detection this should be tried with some faults values. To do such 1000 samples obtained in the same way as for the baseline samples and the test samples, has been obtained for each of the three fault case implementing a small damage value. What has been obtained is the following graph showing the Mahalanobis distance value for each sample, including the undamaged case.

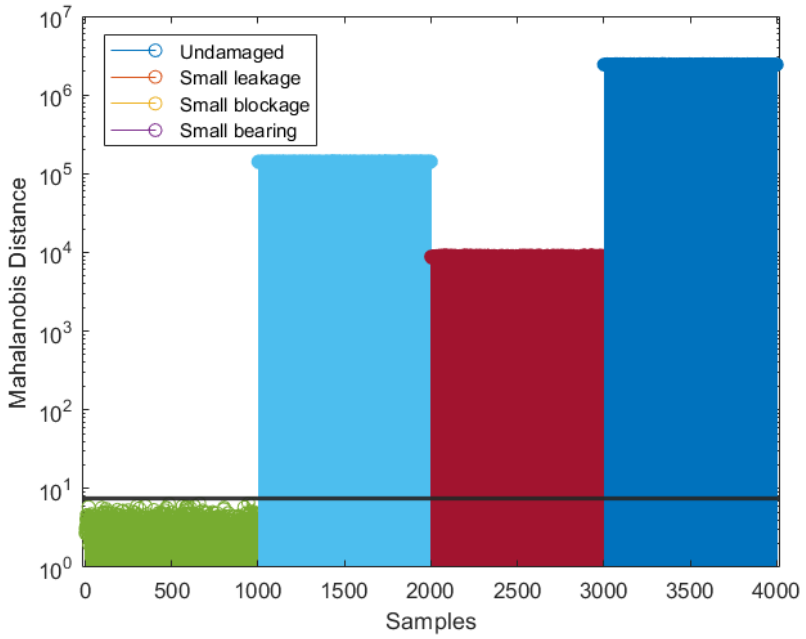

Figure (14): Mahalanobis distance for the undamage case and each damage case

To make the graph clearer the y axis is logarithmic since the compared values has difference of orders of magnitude. It's clear that the anomaly cases has been well classified even with a small damage parameter. Instead some doubt can born if the undamaged cases are considered, indeed from this graph is not clear whether or not someones are misclassified. To better understand the actual performance of the anomaly detection it has been performed, it can be exploited the confusion matrix shown in the next figure.

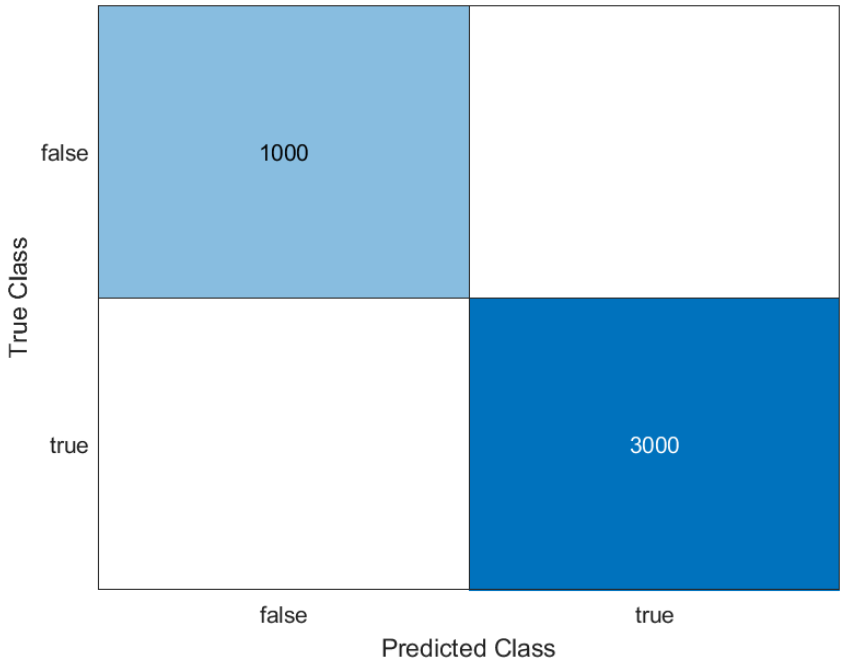

Figure (15): Confusion matrix showing the performances of the anomaly detection

2.2. Fault classification

Once the pump has been classified as "damaged" through anomaly detection performed, it is needed to distinguish the damages that it could have. Eight different possible damage condition are considered:

- Undamaged
- Leakage
- Blockage
- Bearing fault
- Leakage + blockage
- Leakage + bearing fault
- Blockage + bearing fault
- All of them

It is though a classification problem. To perform such classification a Neural Networks needs to be trained for that which means that it has to identify the class instead of doing a regression. Practically what that mean is that giving to the Neural Network as input the features of the signals obtained from the sensor, this should gave as output which class of damage those inputs are associated to.
In order to train the ANN a starting dataset it's needed including all the classes listed above. Actually a very important property of the dataset used is that it needs to be balanced, that is to say every class should have the same number of samples, which is not obvious since exists a lot of combinations with all the damages compared with the other cases, so some of them needs to be randomly chosen among the all set. The risk of having an unbalanced dataset is that the Neural Network trained on it may tend to preferentially give the solution in which there are more cases in the training set, since statistically may be convenient. Lets go more into detail on how the dataset has been created.
For each of the three possible damages ten values has been defined going linearly from the smallest (undamaged) to the biggest damage value. The combination of each of those values gave 1000 different combination of damage condition:

- 1 undamaged condition
- 9 single damage condition for each damage case
- 81 double damage condition for each couple of damage
- 729 cases with all the damages

The given dataset for the activity used all this combination on which for each of them has been applied Gaussian noise for 50 times, in the end though the set dimension was of 50000 samples but unbalanced. To create a balanced dataset it can be used the 50000 sample set and from it choose the needed samples to have then a balanced one. Actually that is a waste of computational resources since not all the performed simulation will be exploited in order to do such. In order to optimize this aspect another solution has been exploited to be also able to have more samples for each class. In particular it has been created a code which creates as many inputs for the simulation as are needed for obtaining a certain predefined amount of samples for each case. This also ensures that for the case in which all the damages are considered a bigger number of combination is taken into account, randomly chosen. In the end, such created dataset has 3600 samples, which means 450 sample for each considered class (the undamaged one is included). In the end the ANN has been trained on this dataset and the performance of it are shown in the confusion matrix presented here following.

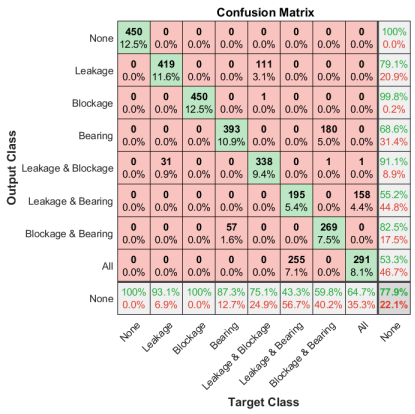

Figure (16): Confusion matrix of the training dataset, single sensor case

Anyway to test the actual performance of the trained ANN it should be tried on some damage value which has not been tested before, in order to understand its interpolation capabilities. To do such other 400 samples has been created, so 50 for each class, including only a value of a medium sized damage not considered before (a mean between the fourth and the fifth value). The results of such test in terms of confusion matrix are shown in the following image.

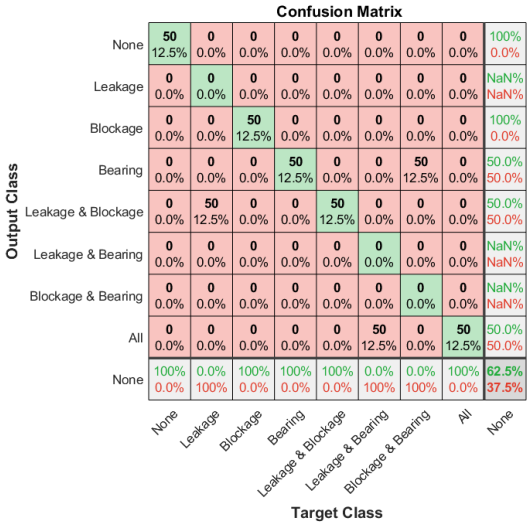

Figure (17): Confusion matrix of the test dataset, single sensor case

As can be seen form the confusion matrices plotted before the classification is poor mostly with the classes involving bearing fault/wear. To improve the classification performances it can be implemented another sensor reading. This practically means that the total available features the Neural Network takes as input are 28 instead of 14 since two signals are considered. This means having more information to identify the class and theoretically this should improve the classification. Obviously the added sensor signal should be related to the damage the Neural Network has more difficulty to classify, so the chosen signal for this purpose is the motor rotational speed.
Practically what has been done in the code was, starting from the same balanced dataset created before, modifying the preprocess part in order to extract the features also of this new signal, after having performed resampling and added Gaussian noise also to it. The Neural Network has then been trained on this new set of data which includes the features of the two sensors. The performance is shown below by the confusion matrix.

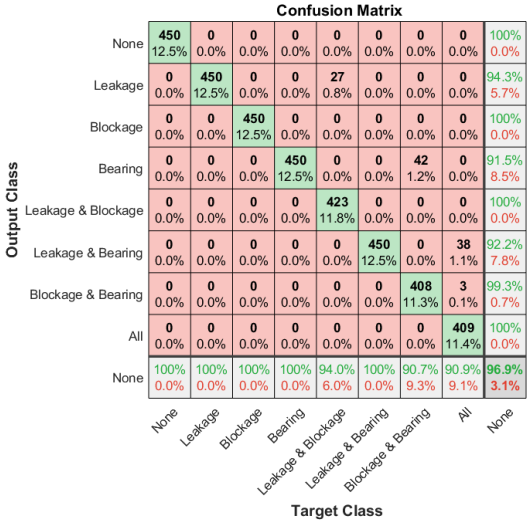

Figure (18): Confusion matrix of the training dataset, double sensor case

As before also a testing including some damage values not included in the training dataset has been performed by using the same set of data used before on this purpose. The interpolation performance can though be seen in the following confusion matrix.

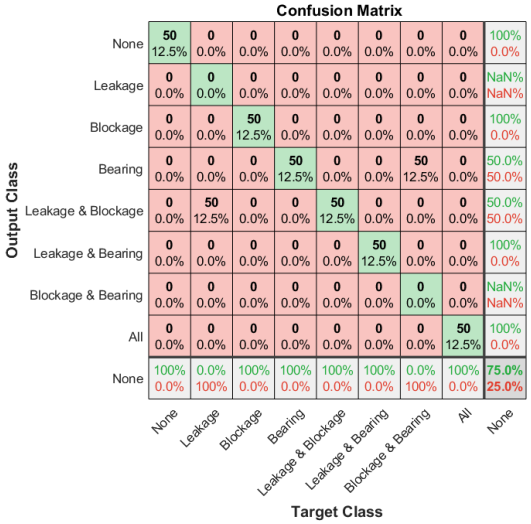

Figure (19): Confusion matrix of the test dataset, double sensor case

As can be seen from the confusion matrices above, with two sensors the classification has improved a lot. Quantitatively with regard to the training set the percentage of right classification passed from 77.9% to 96.9% which is an improvement of about 24%. Anyway things changes if the test dataset with different damage value is considered. There is still an improvement but this time from 62.5% the right classification percentage passes to 75%, which is still a nice improvement but the final performance in case of interpolation is still poor for some classes. Different combination of sensors and number of them may be tried to improve performance, but it requires a lot of time and is not the focus of this activity.

3. Surrogate modeling

In the next paragraph will be needed surrogate models to approximate the Simulink pump model in order to make it faster to run. Anyway how this surrogate are implemented and why will be needed will be better explained in the next paragraph. In this paragraph the focus will be more on describing the actual surrogate modeling procedure.
The surrogate model is a Neural Network trained to fit a direct problem in which, applied to this case, the inputs are the damage parameters and the output is the observed feature, or in this case the signals features. The mathematical formulation for a single neuron consists in a computing cell, shown in the following figure, which performs the following actions:

- Receives information s_i from connected neurons.
- Sums them.
- Return an activation impulse z_j which is modulated through an activation function $h_j(:)$.
- Passes to a consecutive computing unit.

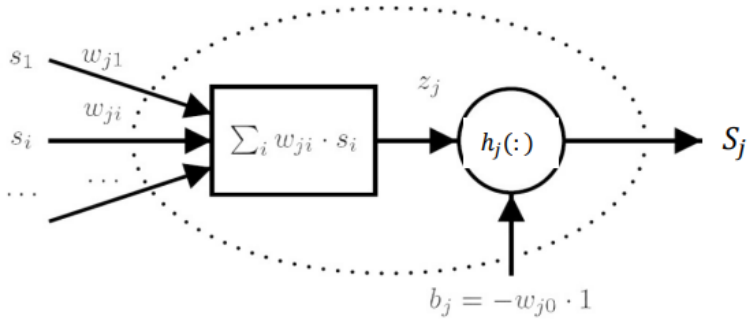

Figure (20): Mathematical single neuron computing cell

Many kinds of Neural Network exists but here it will be used a feed-forward artificial neural network, specifically, Multi-Layer-Perceptrons which is composed by the following layers:

- A input layer in which the input data to be processed are received and distributed to the next layer of neurons.
- A hidden layer (or more than one) which process data from other neurons and provides input for a consecutive layer. They don not comunicate with the outside world.
- An output layer which provides the final output of the network.

This kind of Neural Network structure is shown in the next figure.

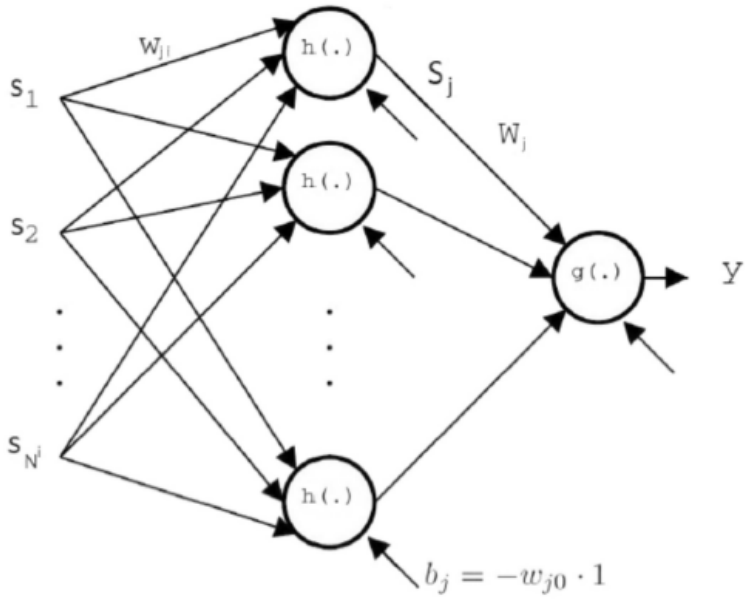

Figure (21): Three layer ANN

In the end the output of the shown artificial neural network structure can be summed up by the following equation:

$$y = g\left(\sum_{j=0}^{N^h} W_j \cdot h\left(\sum_{i=0}^{N^i} w_{ji} s_i\right)\right)$$

Most of the job will be the training of the ANN. In the supervised framework this means using a set of network inputs for which the output is known, and then adjust the weights of the network in order to fit those data. Practically is like any regression problem in which it is needed to minimize an error function by changing the weights parameters, so in the end is an optimization problem. The general algorithm can be though summed as follows:

1. Choose general initial weight vector $\bar{w_1}$, for the first iteration.
2. Determine a search direction $\bar{p_k}$ and a step size η_k so that

$$E(\bar{w}_k + \eta_k \bar{p_k}) < E(\bar{w}_k) \tag{1}$$

3. Update vector $\bar{w}$ with the following update rule

$$\bar{w}_{k+1} = \bar{w}_k + \eta_k \bar{p_k} \tag{2}$$

4. Repeat steps 2-3 while error keeps reducing.

In this context the $\bar{p_k}$ parameter is defined as the gradient of the error function with respect to the weight vector. Because of this it is said that we are in the context of error back-propagation, that is to say the algorithm through the error gradient search "backward" which is the influence of the weight vector on the error. The error equation is the following:

$$E = \sum_{n=1}^{N} (t_n - y_n)^2$$

Where t_n are the target outputs and y_n are the ANN outputs.
A problem that can occur while training the ANN is the overfitting or overtraining. This happens when the Network learn not only the significant features of the problem dealt, but also the noise in the data. This results in increasing the complexity of the ANN and making the training procedure too long.
The way this problem can be solved is dividing the training set in three different categories:

- Training set: used to adjust synapses weights.
- Validation set: used for validation, so at each iteration the error is evaluated on the validation set after weight adjustment. If the error relative to this set increase for a certain predefined amount of times the training is ended.
- Test set: for performance check on data never seen during training.

Usually the division is 70% for the training set, 15% for the validation set and 15% for the test set.

3.1. Surrogate modeling of the triplex reciprocating pump

In the context of this activity the surrogate models are needed to predict the interested signals features from the input damages value. Since the features are 14 for a single signal two possibilities of surrogate modeling arise:

- Training one ANN for each feature, so given in input the damage values will be obtained as output a single feature value. This means that with 14 features will be needed 14 ANNs.
- Training a single ANN which for a given set of damage values gaves in output all the 14 features.

Lets first talk about the first solution. Since it is a regression problem this time the problem of having a balanced dataset is not present, instead the more data are available the better it is, for this reason to train the ANNs will be used the original dataset of 50000 samples. Actually it is important to specify that for the training has been used all the damages value apart from the fourth one for each of the two damage parameters, which will be used for performing the MCMC-MH later. Lets now see what it has been obtained with a 10 neurons ANN. In general checking the regression the performance of the ANNs are good enough for 13 cases over a total of 14. Indeed the "pkurtosis" feature is not well approximated by the ANN. This probably is due to a too low amount of neurons of the ANN which though is not able to approximate well the given data, because is the error evolution graph is considered it can be noticed that it stabilizes to an high value. Might also be that this feature is too much noise dependent and so is not strictly related to the inputs. Anyway, the resulting regression graphs and error evolution are shown in the following figure.

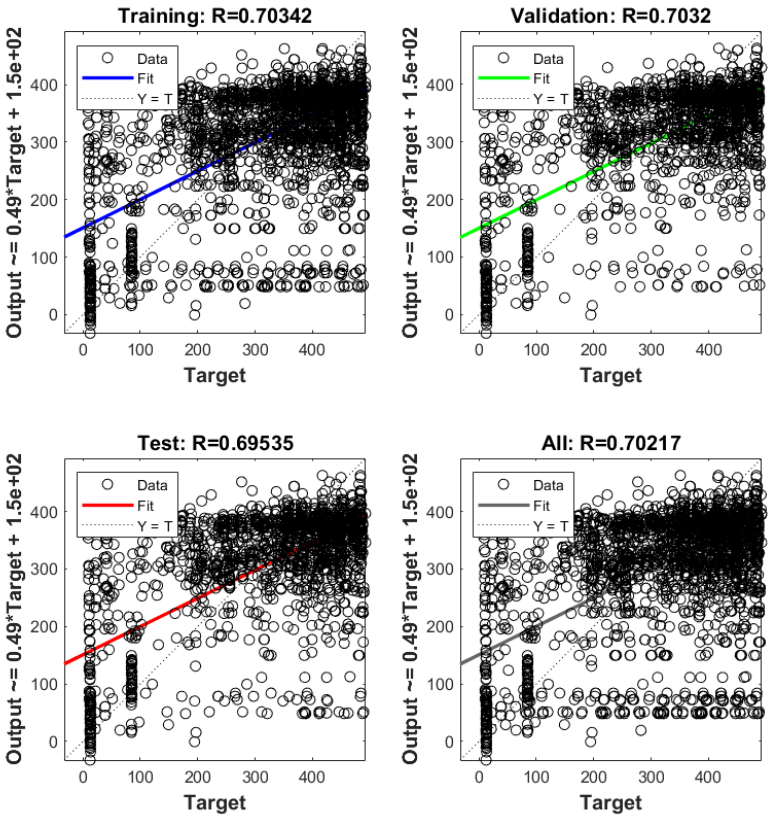

Figure (22): ANN regression for the pKurtosis parameter

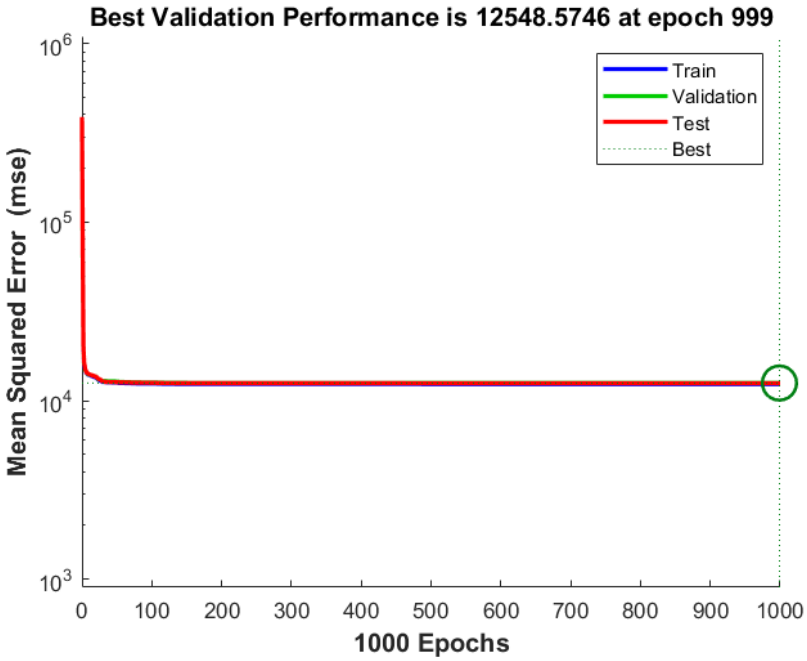

Figure (23): ANN error evolution for the pKurtosis parameter

The second case has also been tried, so training a single multi-output neural network with 10 neurons that gives as output all the features instead of only one. The results in term of regression and error for this ANN are shown here below.

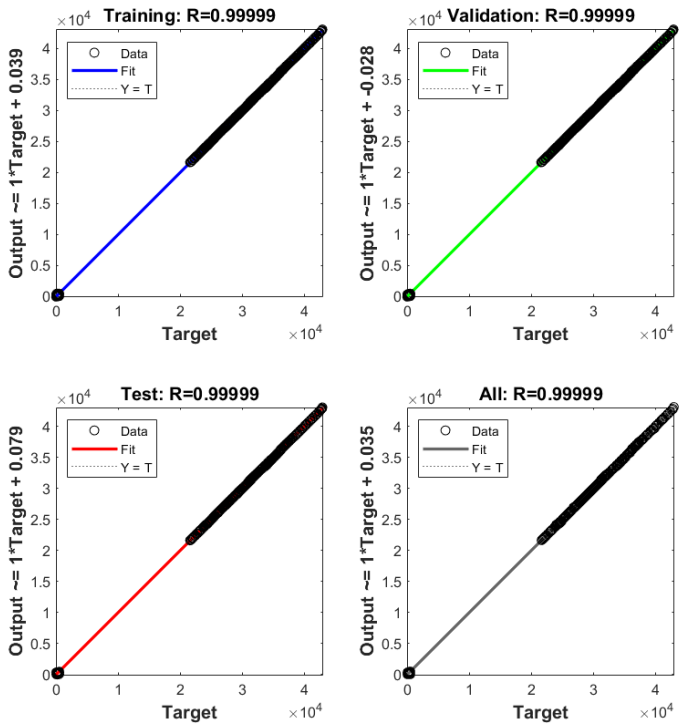

Figure (24): Single multi-output ANN regression

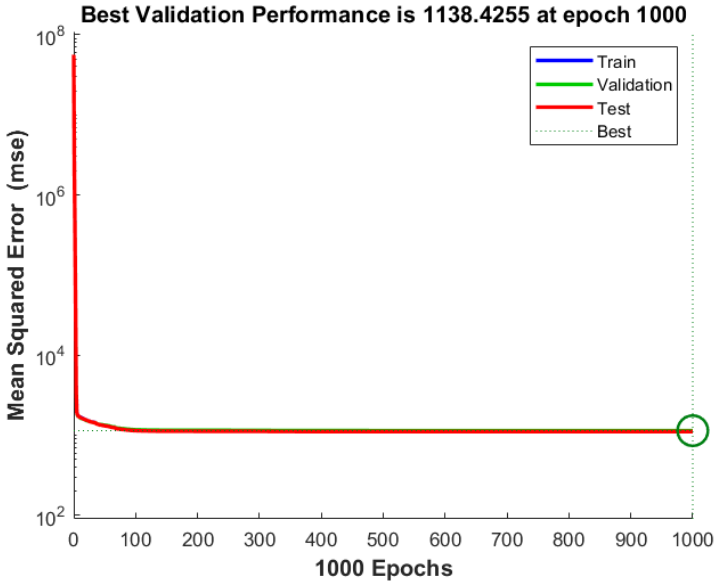

Figure (25): Single multi-output ANN error evolution

Even though the result seems to be nice, it is not as well as the single output ANNs trained before. This because having a relativaly small global error for all the 14 features does not mean that a nice prediction for each of the features taken alone is obtained. To have a quantitative idea of the performances of the two approaches in the next table are shown the standard deviations of the error in the prediction of both the approach ANNs.

	qMean	qVar	qSkew.	qKurt.	qP2P	qCrest	qRMS	qMAD	qCSRange	fPeak	pLow	pMid	pHigh	pKurt.
std 1 out ANN	0.0195	0.0362	0.0108	0.0155	0.0624	0.0014	0.0093	0.0070	36.164	0.6484	0.8403	0.7655	0.4935	NaN
std multiout ANN	0.2637	0.6076	0.0844	0.1431	0.5428	0.0293	0.2547	0.1646	35.476	4.8980	9.8913	13.0758	4.0926	120.217

Table 1: Standard deviation of the features obtained through the ANNs

4. Model Updating by MCMC-MH

Last step of this diagnosis activity related to a triplex reciprocating pump is related to model updating through MCMC-MH. Practically what needs to be done is inferring the damages values starting from the sensor readings. Lets see the procedure to do such.
The used method will be the Markov-Chain Monte-Carlo - Metropolis-Hasting algorithm. The target in general is to perform parameter identification, so an expected value with related variance (damage values), based on observations (sensors readings). This means it is wanted a conditional distribution. Such conditional distribution can be evaluated through the Bayesian approach in which, given that θ is the parameter wanted to be inferred and z is the observation (measure), the Bayes theorem states:

$$p(\theta|z) = \frac{p(z|\theta) \cdot p(\theta)}{p(z)}$$

where:

- $p(\theta|z)$ is the posterior (what is wanted to be found).
- $p(\theta)$ is the prior (supposed to be known).
- $p(z)$ is the prior of the observation which is evaluated as follows:

$$p(z) = \int p(z|\theta) \cdot p(\theta)\, d\theta$$

- $p(z|\theta)$ is the LIKELIHOOD.

The likelihood is evaluated in a Gaussian way as:

$$L(\hat{\vartheta}) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{\frac{-(\hat{z}(\hat{\vartheta})-z)^2}{2\sigma^2}}$$

where σ is the uncertainty of the measure and $z(\bar{\theta})$ is the measure which is likely to have if $\theta = \bar{\theta}$.
The problem of such approach given as is, is that analytical solution are possible only for simple cases and solving on a grid is computationally expensive for problems with lots of parameters. To find the solution to this problem in an efficient way MCMC-MH is used.
The expected parameter can be defined as:

$$E = \int \theta \cdot p(\theta|z)\, d\theta$$

Monte-Carlo sampling allows to approximate it but it is still needed to sample from $p(\theta|z)$ and though the $p(\theta|z)$ distribution. It is though needed an efficient way to sample from $p(\theta|z)$.
On this purpose lets introduce the Markov-Chain. Practically is a sequence of random samples that have the Markov property: the probability of moving to the next state depends only on the present one:

$$p(\theta_{k+1}|\theta_k, ..., \theta_1) = p(\theta_{k+1}|\theta_k)$$

The "random walk" created in this way needs to be stable (stationarity). Stationarity is reached sufficiently if Reversibility property is satisfied (detailed balance condition):

$$p(\theta_{k+1}|\theta_k) \cdot p(\theta_k) = p(\theta_k|\theta_{k+1}) \cdot p(\theta_{k+1})$$

In this relation the two terms $p(\theta_{k+1}|\theta_k)$ and $p(\theta_k|\theta_{k+1})$ are difficult to be found, and $p(\theta_{k+1})$ can be also modified as $p(\theta_{k+1}|x)$.
To find those terms the Metropolis-Hasting approach can be exploited. It consists of writing a distribution as a proposal distribution multiplied by an acceptance ratio:

$$p(\theta_{k+1}|\theta_k) = q(\theta_{k+1}|\theta_k) \cdot \alpha(\theta_{k+1}|\theta_k)$$

The proposal distribution is user defined and the standard deviation of it needs to be adjusted in order to have a good performance of the algorithm. The acceptance ratio instead, recalling the Bayes theorem and assuming a symmetric proposal, is defined as:

$$\alpha(\vartheta_{k+1}|\vartheta_k) = \min\left(\frac{\mathcal{L}(x|\vartheta_{k+1}) \cdot p(\vartheta_{k+1})}{\mathcal{L}(x|\vartheta_k) \cdot p(\vartheta_k)}, 1\right)$$

So summing up, in the end the process to build the Markov-Chain through the Metropolis-Hasting algorithm will be:

1. Draw a tentative sample according to $q(\theta_{k+1}|\theta_k)$.
2. Accept it based on $\alpha(\theta_{k+1}|\theta_k)$.

Once such is done what will results is a discrete $p(\theta|z)$ (histogram) or better, samples from such distribution. Applying Monte-Carlo to approximate the expected value for θ will result in:

$$E(\theta) = \frac{1}{L}\sum_{i=1}^{N}\theta_i$$

There is still a problem in terms of computational cost. To evaluate $\alpha(\theta_{k+1}|\theta_k)$ for each iteration requires to derive the likelihood which requires itself $z(\bar{\theta})$. This last term usually needs a model describing the system to run, which is expensive for complex systems. To solve that issue instead of directly using the model, it can be used a surrogate model which approximate it and which has already been described in the previous paragraph.

4.1. Application of MCMC-MH to the triplex reciprocating pump

All the process shown now needs to be applied to the pump case study. The sensor readings considered for this part of the activity will be the one relative to the output flow rate. Actually what will be used, as always in this activity, are the features of such signal. Theoretically the damage parameters to be inferred are three, but for simplicity only two will be considered and those are:

- Leakage area.
- Blockage factor.

However there is a problem, which is the two considered parameters have order of magnitudes of difference between themselves. A scaling factor of 10^7 needs to be applied on the leakage area parameter in order for them to be comparable.
The prior then needs to be defined. A priori it can be assumed that the two parameters are uncorrelated thus the following distributions can be used:

- Leakage size * 10^7: Weibull distribution, scale factor 15, shape factor 1.5.
- Blockage factor: Beta distribution, first shape parameter 4, second shape parameter 1.2.

The prior distribution shape looks like in the following image.

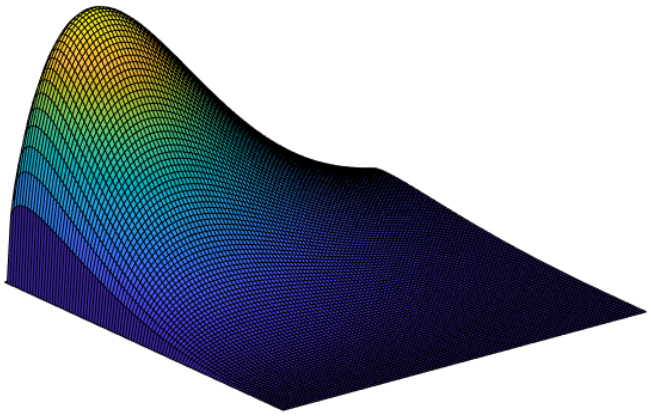

Figure (26): Prior distribution shape

For the likelihood a Gaussian likelihood is used in which the measurements are assumed to be uncorrelated. With regard to the measurement uncertainty to be used, both the measure noise (signal noise) and the discrepancy between surrogate model and true model needs to be considered. This last uncertainty term is the one shown in the Surrogate modeling section.

With regard to the surrogate model approach used, it has been applied the ANNs trained for a single feature. Actually since, as seen in the previous paragraph, one of those had poor performances only 13 of them and so 13 features has been considered.
As a proposal a multivariate normal distribution has been chosen with the following covariance matrix:

$$Cov = \begin{bmatrix} (0.1)^2 & 0 \\ 0 & (2 \times 0.01)^2 \end{bmatrix}$$

With this setup the acceptance ratio obtained while preforming the MCMC-MH falls around the range of 0.4-0.5, which is the optimal one. The obtained Markov-Chain from the algorithm is shown in the next figure.

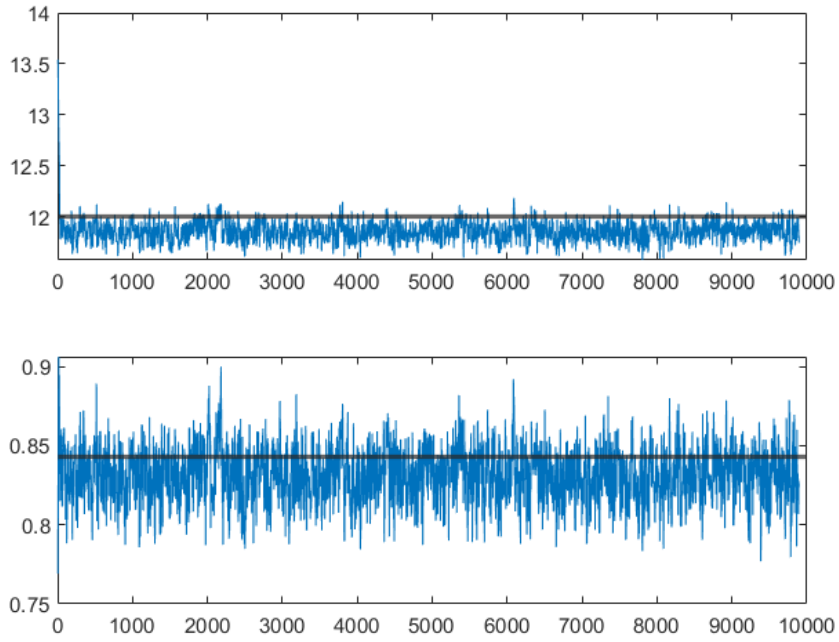

Figure (27): Markov-Chain "Random walk" obtained through Metropolis-Hasting algorithm

Obtained this, it needs to be processed by removing the burn-in period and by thinning. Removing the burn-in period means to remove the first part of the chain since it is highly influenced by the initial conditions (initial guess). Thinning instead consists in keeping one sample every N samples in order to remove dependencies among consecutive samples The resulting discrete distributions from the processed Markov-Chain, compared to the true values to be inferred, is represented in the following images.

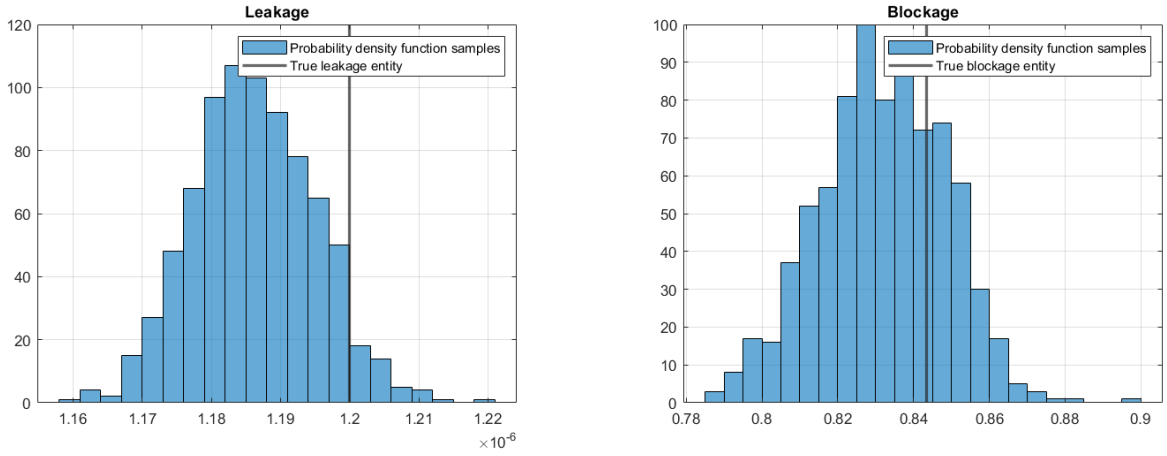

Figure (28): Discrete posteriors obtained for the leakage are on the left and for the blockage factor on the right

The mean values and standard deviation of such distributions are summed up in the following table.

	mean	**std**
Leak area	1.19×10^{-6}	8.79×10^{-9}
Blocking factor	0.831811	0.016691

Table 2: Mean value and standard deviation of leak are and block factor parameters

Here following, to complete the results obtained from the algorithm, is shown the bi-variate samples distribution coming form the processed Markov-Chain.

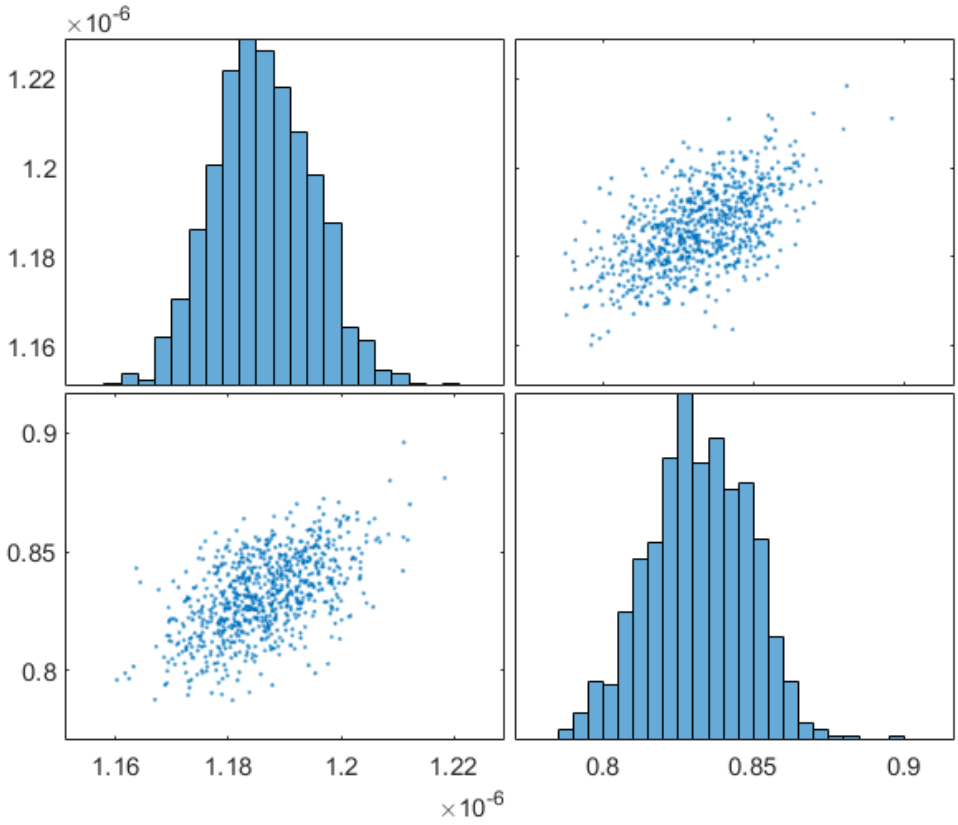

Figure (29): Bi-variate samples distribution

It can be seen that the error on the prediction for the leakage area is about 55%, instead for the blockage is around 1%. If this error is considered relatively to the damages values ranges for the leakage are the deviation is about 18%, instead for the blockage factor is 2%, so anyway the prediction on the blockage factor is more accurate than the one obtained for the leakage area.

Conclusions

What has been done in this activity was a diagnostic activity over a triplex reciprocating pump. In order to have data about such pump a Simulink model, which have been described in this report, have been used. In such model were included tree different damage types.
The process steps for the pump diagnosis has been the following:

- Anomaly detection.
- Fault classification.
- Model updating by MCMC-MH, and by using surrogate models.

Anomaly detection has been performed with success and the implemented procedure shows nice performances. The fault classification has been done both with one sensor and two sensors, comparing then the obtained results, which actually demonstrated that such as done classification is not very reliable.
The model update process through MCMC-MH has given better results for one of the two damages considered but anyway a good overall result has been obtained.
For the surrogate modeling activity two different approaches has been tried and compared and, in the end, the best one has been used for its purpose.
In the end in this practice have been successfully performed the all the steps of a diagnosis activity over a reciprocating pump.

Appendices

For the codes here given to work they need to be plugged in the respective script.

Appendix 1

Here is the Matlab code used for creating the balanced dataset for the two sensors case (single sensor is similar

```
%% Build the dataset that spans all the possible damage configurations

% create a starting random balance dataset
leakarea_noundamaged_set = leak_area_set(2:10);
blockingfactor_noundamaged_set = blockinfactor_set(2:10);
bearingfactor_noundamaged_set = bearingfactor_set(2:10);
[LeakGrid,BlockedInputGrid] = ndgrid(leakarea_noundamaged_set,
    blockingfactor_noundamaged_set);
leak_blocking_set = [LeakGrid(:), BlockedInputGrid(:)];
[LeakGrid,BearingFactorGrid] = ndgrid(leakarea_noundamaged_set,
    bearingfactor_noundamaged_set);
leak_bearing_set = [LeakGrid(:), BearingFactorGrid(:)];
[BlockedInputGrid,BearingFactorGrid] = ndgrid(blockingfactor_noundamaged_set,
    bearingfactor_noundamaged_set);
blocking_bearing_set = [BlockedInputGrid(:), BearingFactorGrid(:)];
[LeakGrid, BlockedInputGrid, BearingFactorGrid] = ndgrid(leakarea_noundamaged_set,
    blockingfactor_noundamaged_set, bearingfactor_noundamaged_set);
alldamage_set = [LeakGrid(:), BlockedInputGrid(:), BearingFactorGrid(:)];

sz = size(leakarea_noundamaged_set,2);
numRepetitions = 50;
set_dimension = numRepetitions*sz;

nodamage_set = [leak_area_set(1), blockinfactor_set(1), bearingfactor_set(1)];

if size(leak_blocking_set,1) > set_dimension
    leak_blocking_selected = leak_blocking_set(randperm(size(leak_blocking_set,1),
        set_dimension),:);
    leak_blocking_selected(:,3) = ones(set_dimension,1)*bearingfactor_set(1);

    leak_bearing_selected = leak_bearing_set(randperm(size(leak_bearing_set,1),
        set_dimension),:);
    leak_bearing_selected(:,3) =  leak_bearing_selected(:,2);
    leak_bearing_selected(:,2) = ones(set_dimension,1)*blockinfactor_set(1);

    blocking_bearing_selected = blocking_bearing_set(randperm(size(
        blocking_bearing_set,1),set_dimension),:);
    blocking_bearing_selected(:,2:3) = blocking_bearing_selected;
    blocking_bearing_selected(:,1) = ones(set_dimension,1)*leak_area_set(1);
else
    leak_blocking_selected = leak_blocking_set;
    leak_blocking_selected(:,3) = ones(size(leak_blocking_selected,1),1)*
        bearingfactor_set(1);

    leak_bearing_selected = leak_bearing_set;
    leak_bearing_selected(:,3) =  leak_bearing_selected(:,2);
    leak_bearing_selected(:,2) = ones(size(leak_bearing_selected,1),1)*
        blockinfactor_set(1);

    blocking_bearing_selected = blocking_bearing_set;
    blocking_bearing_selected(:,2:3) = blocking_bearing_selected;
    blocking_bearing_selected(:,1) = ones(size(blocking_bearing_selected,1),1)*
        leak_area_set(1);
end

if size(alldamage_set,1) > set_dimension
    alldamage_selected = alldamage_set(randperm(size(alldamage_set,1),set_dimension)
```

```
        ,:);
else
    alldamage_selected = alldamage_set;
end

MatGrid = [nodamage_set;...
    leakarea_noundamaged_set', ones(sz,1)*blockinfactor_set(1), ones(sz,1)*
        bearingfactor_set(1);...
    ones(sz,1)*leak_area_set(1), blockingfactor_noundamaged_set', ones(sz,1)*
        bearingfactor_set(1);...
    ones(sz,1)*leak_area_set(1), ones(sz,1)*blockinfactor_set(1),
        bearingfactor_noundamaged_set';...
    leak_blocking_selected;...
    leak_bearing_selected;...
    blocking_bearing_selected;...
    alldamage_selected];

% We are gonna use Simulink.SimulationInput objects
for ct = size(MatGrid, 1):-1:1
    simInput(ct) = Simulink.SimulationInput(mdl);
    simInput(ct) = setVariable(simInput(ct),'leak_cyl_area_WKSP', MatGrid(ct, 1));
    simInput(ct) = setVariable(simInput(ct),'block_in_factor_WKSP',MatGrid(ct, 2));
    simInput(ct) = setVariable(simInput(ct),'bearing_fault_frict_WKSP',MatGrid(ct, 3)
        );
    simInput(ct) = setVariable(simInput(ct),'noise_seed_offset_WKSP',ct-1);
end


[ok,e] = generateSimulationEnsemble(simInput,fullfile('.','Data'),'UseParallel',false
    );

ens = simulationEnsembleDatastore(fullfile('.','Data'));

% After having run the simulations, we want to extract the features from
% each simulation and the corresponding values of the damage parameters.
% To do that, at first we configure the ensemble variables, and then we
% loop over every simulation, adding different noise samples for some
% number of repetitions.

% Configure ensemble variables
% Append the features (or Condition Indicators) that we compute with the
% function "extractCI" to the datastore
ens.DataVariables = [ens.DataVariables; ...
    "fPeak1"; "pLow1"; "pMid1"; "pHigh1"; "pKurtosis1"; ...
    "qMean1"; "qVar1"; "qSkewness1"; "qKurtosis1"; ...
    "qPeak2Peak1"; "qCrest1"; "qRMS1"; "qMAD1"; "qCSRange1";...
    "fPeak2"; "pLow2"; "pMid2"; "pHigh2"; "pKurtosis2"; ...
    "qMean2"; "qVar2"; "qSkewness2"; "qKurtosis2"; ...
    "qPeak2Peak2"; "qCrest2"; "qRMS2"; "qMAD2"; "qCSRange2"];
% Store the Condition Variables Names in the datastore
ens.ConditionVariables = ["LeakFault","BlockingFault","BearingFault"];
% Read one element from the ensemble

data = read(ens);
% Extract the fault features
[flow1,flowSpectrum1,flowFrequencies1,flow2,flowSpectrum2,flowFrequencies2,
    faultValues] = preprocess2(data);
feat = extractCI2(flow1,flowSpectrum1,flowFrequencies1,flow2,flowSpectrum2,
    flowFrequencies2);
% Append the fault values to the features and store them in the
% dataToWrite Cell
dataToWrite = [faultValues, feat]; %, {'Frequency', flowF, 'Power', flowP}];


% Loop over all members in the ensemble, preprocess
% and compute the features for each member.
```

```
106
107 reset(ens)
108 % Set the number of files that are acquired whenever the read method is
109 % called
110 ens.ReadSize = 1;
111 % Select which variables to read from the ensemble files
112 % ens.SelectedVariables = ["SimulationInput", "qOut"];
113 % Preallocate table
114 DATA = table('Size', [8*sz*numRepetitions, length(dataToWrite)/2], ...
115     'VariableTypes', [repmat("double", 1, length(dataToWrite)/2)],...
116     'VariableNames', dataToWrite(cellfun(@(x) ischar(x), dataToWrite)));
117 appendCounter = 0;
118 ct = 0;
119 numRepetitions_set2 = round(set_dimension/size(blocking_bearing_selected,1),
        TieBreaker="plusinf");
120 numRepetitions_set3 = round(set_dimension/size(alldamage_selected,1),TieBreaker="
        plusinf");
121 ct_set21 = 0;
122 ct_set22 = 0;
123 ct_set23 = 0;
124 ct_set3 = 0;
125
126 while hasdata(ens)
127
128     % Read member data
129     data = read(ens);
130
131     class = dataClass(data,leak_area_set(1),blockinfactor_set(1),bearingfactor_set(1)
            );
132     switch class
133
134         case 0
135             for counter = 1:set_dimension
136                 % Preprocess and extract features from the member data
137                 [flow1,flowP1,flowF1,flow2,flowP2,flowF2,faultValues] = preprocess2(
                        data);
138                 feat = extractCI2(flow1,flowP1,flowF1,flow2,flowP2,flowF2);
139                 % Add the extracted feature values to the member data
140                 dataToWrite = [faultValues, feat];
141                 appendCounter = appendCounter + 1;
142                 appendCounterVariable = 0;
143                 for counterVariable = 1:2:length(dataToWrite)
144                     appendCounterVariable = appendCounterVariable + 1;
145                     DATA{appendCounter, appendCounterVariable} = dataToWrite{
                            counterVariable + 1};
146                 end
147             end
148
149         case 1
150             for counter = 1:numRepetitions
151                 % Preprocess and extract features from the member data
152                 [flow1,flowP1,flowF1,flow2,flowP2,flowF2,faultValues] = preprocess2(
                        data);
153                 feat = extractCI2(flow1,flowP1,flowF1,flow2,flowP2,flowF2);
154                 % Add the extracted feature values to the member data
155                 dataToWrite = [faultValues, feat];
156                 appendCounter = appendCounter + 1;
157                 appendCounterVariable = 0;
158                 for counterVariable = 1:2:length(dataToWrite)
159                     appendCounterVariable = appendCounterVariable + 1;
160                     DATA{appendCounter, appendCounterVariable} = dataToWrite{
                            counterVariable + 1};
161                 end
162             end
163
164         case 2
```

```
            if ct_set21 < 450
                for counter = 1:numRepetitions_set2
                    % Preprocess and extract features from the member data
                    [flow1,flowP1,flowF1,flow2,flowP2,flowF2,faultValues] =
                        preprocess2(data);
                    feat = extractCI2(flow1,flowP1,flowF1,flow2,flowP2,flowF2);
                    % Add the extracted feature values to the member data
                    dataToWrite = [faultValues, feat];
                    appendCounter = appendCounter + 1;
                    appendCounterVariable = 0;
                    for counterVariable = 1:2:length(dataToWrite)
                        appendCounterVariable = appendCounterVariable + 1;
                        DATA{appendCounter, appendCounterVariable} = dataToWrite{
                            counterVariable + 1};
                    end
                    ct_set21 = ct_set21 + 1;
                end
            end

        case 3
            if ct_set22 < 450
                for counter = 1:numRepetitions_set2
                    % Preprocess and extract features from the member data
                    [flow1,flowP1,flowF1,flow2,flowP2,flowF2,faultValues] =
                        preprocess2(data);
                    feat = extractCI2(flow1,flowP1,flowF1,flow2,flowP2,flowF2);
                    % Add the extracted feature values to the member data
                    dataToWrite = [faultValues, feat];
                    appendCounter = appendCounter + 1;
                    appendCounterVariable = 0;
                    for counterVariable = 1:2:length(dataToWrite)
                        appendCounterVariable = appendCounterVariable + 1;
                        DATA{appendCounter, appendCounterVariable} = dataToWrite{
                            counterVariable + 1};
                    end
                    ct_set22 = ct_set22 + 1;
                end
            end


        case 4
            if ct_set23 < 450
                for counter = 1:numRepetitions_set2
                    % Preprocess and extract features from the member data
                    [flow1,flowP1,flowF1,flow2,flowP2,flowF2,faultValues] =
                        preprocess2(data);
                    feat = extractCI2(flow1,flowP1,flowF1,flow2,flowP2,flowF2);
                    % Add the extracted feature values to the member data
                    dataToWrite = [faultValues, feat];
                    appendCounter = appendCounter + 1;
                    appendCounterVariable = 0;
                    for counterVariable = 1:2:length(dataToWrite)
                        appendCounterVariable = appendCounterVariable + 1;
                        DATA{appendCounter, appendCounterVariable} = dataToWrite{
                            counterVariable + 1};
                    end
                    ct_set23 = ct_set23 + 1;
                end
            end

        case 5
            if ct_set3 < 450
                for counter = 1:numRepetitions_set3
                    % Preprocess and extract features from the member data
                    [flow1,flowP1,flowF1,flow2,flowP2,flowF2,faultValues] =
                        preprocess2(data);
```

```
                    feat = extractCI2(flow1,flowP1,flowF1,flow2,flowP2,flowF2);
                    % Add the extracted feature values to the member data
                    dataToWrite = [faultValues, feat];
                    appendCounter = appendCounter + 1;
                    appendCounterVariable = 0;
                    for counterVariable = 1:2:length(dataToWrite)
                        appendCounterVariable = appendCounterVariable + 1;
                        DATA{appendCounter, appendCounterVariable} = dataToWrite{
                            counterVariable + 1};
                    end
                    ct_set3 = ct_set3 + 1;
                end
            end

    end
    ct = ct + 1;
    fprintf('Features extracted for simulation #%d\n', ct)
end

save('DATA_2sens.mat',"DATA")
```

Appendix 2

Here is reported the Matlab code used to train the single ANN for the MCMC-MH parameter updating.

```
%% Single ANN training

surrogateModel_build.Leakage = any(DATA.LeakFault == leak_area_set([1:3, 5:10]), 2);
surrogateModel_build.Blockage = any(DATA.BlockingFault == blockinfactor_set([1:3,
    5:10]), 2);
surrogateModel_build.Bearing = any(DATA.BearingFault == bearingfactor_set([1:3,
    5:10]), 2);


surrogateModel_test.LeakageFault = leak_area_set(4);
surrogateModel_test.BlockingFault = blockinfactor_set(4);
surrogateModel_test.BearingFault = bearingfactor_set(4);
surrogateModel_test.Leakage = any(DATA.LeakFault == surrogateModel_test.LeakageFault,
     2);
surrogateModel_test.Blockage = any(DATA.BlockingFault == surrogateModel_test.
    BlockingFault, 2);
surrogateModel_test.Bearing = any(DATA.BearingFault == surrogateModel_test.
    BearingFault, 2);

input = [DATA.LeakFault(surrogateModel_build.Leakage | surrogateModel_build.Blockage
    | surrogateModel_build.Bearing), ...
        DATA.BlockingFault(surrogateModel_build.Leakage | surrogateModel_build.
            Blockage | surrogateModel_build.Bearing), ...
        DATA.BearingFault(surrogateModel_build.Leakage | surrogateModel_build.
            Blockage | surrogateModel_build.Bearing)];

output =  DATA{surrogateModel_build.Leakage | surrogateModel_build.Blockage |
    surrogateModel_build.Bearing, 4:17};

% Create a Fitting Network
hiddenLayerSize = 10;
net = fitnet(hiddenLayerSize,'trainlm');

% Setup Division of Data for Training, Validation, Testing
net.divideParam.trainRatio = 70/100;
net.divideParam.valRatio = 15/100;
net.divideParam.testRatio = 15/100;

% Train the Network
[net,tr] = train(net,input',output');
```

```
ANN = net;
% Test the Network
y = net(input');

temp = surrogateModel_test.Leakage | surrogateModel_test.Blockage |
    surrogateModel_test.Bearing;
sigmaSurrogate_singleANN = std(DATA{temp, 4:17} - net(DATA{temp, 1:3}'));
save('sigmaSurrogate_singleANN.mat', 'sigmaSurrogate_singleANN')
if ~exist('./ANN_oneFunction_proj/', 'dir')
    mkdir('./ANN_oneFunction_proj/')
end

genFunction(ANN, sprintf('./ANN_oneFunction_proj/ANN_single.m'))
```